LoTRS: Practical Post-Quantum Structured Threshold Ring Signatures from Lattices

Artifact Appendix

Nikai Jagganath*
nikai.jagganath@monash.edu
Monash University
CSIRO's Data61
Melbourne, Victoria, Australia

Muhammed F. Esgin
muhammed.esgin@monash.edu
Monash University
Melbourne, Victoria, Australia

Ron Steinfeld
ron.steinfeld@monash.edu
Monash University
Melbourne, Victoria, Australia

Amin Sakzad
amin.sakzad@monash.edu
Monash University
Melbourne, Victoria, Australia

Markku-Juhani O. Saarinen
markku-juhani.saarinen@tuni.fi
Tampere University
Tampere, Finland

Dongxi Liu
dongxi.liu@data61.csiro.au
CSIRO Technology
Sydney, New South Wales, Australia

A Artifact Appendix

A.1 Abstract

The LoTRS artifact contains Python and Rust implementations of the concrete $\kappa = 1$ structured threshold ring signature scheme, deterministic cross-language test vectors, a SageMath parameter estimator, and a Rust benchmark harness with data and plotting scripts. It exercises key generation, signing, verification, and serialization; reproduces analytic size and attack-cost estimates; and measures threshold, ring-signature, and multi-signature configurations. LoTRS sources are available under the MIT license; bundled dependencies retain their upstream license notices.

A.2 Description & Requirements

Evaluation instructions are in `README.md`. The `lotrs-py/` directory contains the reference implementation and fixtures; `lotrs-rs/` contains Rust code and benchmarks; `estimator/` contains Sage scripts and bundled estimators. `parameters.json` defines the shared production profile. `lotrs-rs/bench-out/` contains the measured grid, CSV, summary, and five PDF plots. A separate sampler specification describes the FACCT-style large-width Gaussian sampler.

A.2.1 Security, privacy, and ethical concerns. Experiments use synthetic keys, messages, and public deterministic seeds; they require no personal data or external targets. Execution writes local build products and reports. The signing harness is a research prototype: sampling and rejection logic are data-dependent, and the centralized seed interface is not a distributed key-management system. Test keys and seeds must not be used to protect real data.

A.2.2 How to access. **Archive DOI and release identifier: pending final deposit.** The artifact is packaged as source code with fixtures, benchmark outputs, and evaluation instructions. The archival release identifier will select the exact package evaluated. No artifact badge is asserted here.

A.2.3 Hardware dependencies. No specialized hardware is required. The reference machine is an x86-64 AMD Ryzen AI 9 HX 370 with 12 cores, 24 hardware threads, and 32 GB RAM. Rust uses the default Rayon thread pool. Timing comparisons should use comparable hardware and record the available threads. Reserve 5 GB disk space for the artifact, build, and outputs, in addition to SageMath.

A.2.4 Software dependencies. Tested on Linux. Toolchains are Python 3.14.7, Rust/Cargo 1.98.1, and SageMath 10.9. Python dependencies are NumPy 2.5.3, PyCryptodome 3.23.0, and mpmath 1.4.1. Plots use Matplotlib 3.11.1. A C toolchain, Make, and a shell are needed for installation/build steps. Cargo declares the Rust dependencies. Network access is needed to install dependencies; the experiments then run locally without network services.

A.2.5 Benchmarks. Inputs are generated from explicit seeds. The 14-cell grid has $N = 32$ with $T \in \{1, 4, 8, 16, 32\}$, $N = 100$ with $T \in \{1, 5, 10, 25, 50\}$, and $N = 1$ with $T \in \{2, 4, 8, 16\}$. All cells retain the same $d = 128$ lattice and $\phi = \max(22T, 1100)$. Each cell uses 100 signing seeds and 100 verification calls. Verification repeatedly checks that cell's final signature; wire size also refers to that signature. Key generation averages four calls; aggregation is measured once.

A.3 Set Up

A.3.1 Installation. Unpack the release and install Rust, Make, and SageMath, with `cargo` and `sage` on `PATH`. From the artifact root:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install \
  -r lotrs-py/requirements.txt matplotlib
cargo fetch --manifest-path lotrs-rs/Cargo.toml
```

Allow approximately 15 person-minutes, excluding toolchain installation. Component READMEs give further build and command-line details.

A.3.2 Basic test. With the Python environment active, run from the artifact root:

```
python lotrs-py/check_parameters.py
cargo test --release --manifest-path \
  lotrs-rs/Cargo.toml --test parameters
make -C estimator test
```

*Corresponding author.

Expected results are the message "Artifact manifest and Python profiles agree; parameter conditions pass.", one passing Rust parameter test, and five passing Sage tests. Allow five compute-minutes, including initial compilation on the reference machine.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1) The concrete protocol implementation is functional and Python/Rust compatible at the serialization boundary (Section 6.1; experiment E1).

(C2) The analytic signature size is 52.6574 KiB, the public key is 9.40625 KiB, and the signing-repetition heuristic is 4.7728 (Table 3; E2). The bundled attack-cost outputs do not substantiate 128-bit post-quantum security; this is not a claim established by the artifact.

(C3) At $N = 100, T = 50$, the reference benchmark produces a 53.53 KiB signature, with mean signing time 923.6 ms and mean verification time 311.0 ms. At $N = 32, T = 16$, these are 42.72 KiB, 151.4 ms, and 54.56 ms (Table 4 and Appendix F; E3).

A.4.2 Experiments. Time allowances below assume completed installation and the reference machine; they are approximate elapsed times, not core-hours.

(E1) **Conformance (C1).** 5 person-minutes + 5 compute-minutes. From the artifact root with the environment active, run `make check-full`. Expect 142 Python tests, 65 Rust tests including the explicitly enabled production signing test, and five Sage tests to pass. Both signature fixtures and sampler vectors must match exactly. Malformed-signature and norm-bound tests must reject invalid inputs.

(E2) **Estimates (C2).** 5 person-minutes + 20 compute-minutes. Run:

```
make -C estimator reference OUT=estimate.txt
```

In `estimator/`, compare `estimate.txt` against `LoTRS-Estimate-Output-N100T50.txt`. Check C2's sizes to four decimal places and repetitions to four decimal places; ignore incidental paths and runtime fields. ASIS variant 0 reports 100.7978 bits for the binary proof and 94.16636 for DualMS; variants 1 and 2 report 88.86122 and 86.20864 respectively. These are model outputs, not validated security levels.

(E3) **Performance (C3).** 10 person-minutes + 15 compute-minutes. With the Python environment active, run:

```
cd lotrs-rs
mkdir -p bench-evaluation
cargo run --release --example bench -- \
  --grid \
  '32,1:4:8:16:32;100,1:5:10:25:50;1,2:4:8:16' \
  > bench-evaluation/grid-d128.md
python scripts/plot_bench.py \
  bench-evaluation/grid-d128.md
```

Expect 14 result rows and five PDF plots. Compare the generated summary and CSV with `bench-out/`. All verifications must succeed; deterministic wire sizes should match to displayed precision. For comparable 24-thread hardware, use a suggested 20% tolerance for headline timing means; other hardware requires reporting its configuration and measured timings. At $N = 100, T = 50$, the measured attempt mean is 5.28 with standard error 0.50. The CSV inherits rounded report values, so its byte columns must not be treated as exact unrounded lengths.

A.5 Remark on 128-bit Security

Following the methodology of earlier work[1], we choose parameters so that the best attack on each underlying hard problem (MLWE, MSIS and ASIS) requires a root Hermite factor (RHF) of $\delta_0 \approx 1.0045$, as computed by our parameter scripts. The lattice estimator[2] maps an RHF to a BKZ block size β via Chen's formula

$$\delta_0 = \left(\frac{\beta}{2\pi e}(\pi\beta)^{1/\beta}\right)^{1/(2(\beta-1))},$$

so $\delta_0 = 1.0045$ gives $\beta = 335$. Under the estimator's default cost model (MATZOV), BKZ-335 in dimension $d = 670$ costs about $2^{123.3}$ operations. We also ran the estimator on LWE instances whose best primal attack needs $\delta_0 \approx 1.0045$, with modulus $q = 3329$, centred binomial secret and error ($\eta = 2$), and $m = n$ samples:

n	δ_0 (uSVP)	β	uSVP	Dual-hybrid
456	1.00454	330	$2^{122.6}$	$2^{119.0}$
464	1.00447	338	$2^{124.8}$	$2^{121.2}$

These results can be reproduced in SageMath with:

```
from estimator import *
print(RC.MATZOV(RC.beta(1.0045),670).log2())
X = ND.CenteredBinomial(2)
for n in (456, 464):
   LWE.estimate(LWE.Parameters(n=n, q=3329, Xs=X, Xe=X, m=n))
```

Hence, according to the estimator, $\delta_0 = 1.0045$ gives roughly 2^{120}–2^{124}, within 10 bits of 128-bit security. As the accuracy of current cost models is arguably no better than about ±10 bits, we consider $\delta_0 \approx 1.0045$ appropriate for 128-bit security. Core-SVP-style estimates, as in our ASIS scripts, give lower figures: for $\beta = 335$, our ASIS script reports a quantum Core-SVP cost of $2^{0.2653\beta} \approx 2^{88.9}$. However, Core-SVP counts only the exponential cost of a single SVP call in dimension β, ignoring the number of calls made by BKZ and all polynomial factors, and is therefore generally regarded as an underestimate of the real attack complexity.

A.6 Notes on Reusability

The Python API, explicit seeds, shared manifest, and cross-language fixtures support independent implementations and parameter experiments. The benchmark's `-grid` option varies N, T. Parameter changes require rechecking conditions, attack estimates, and sampler support; small correctness-test profiles carry no security claim. The $N = 1$ measurements retain binary-proof scaffolding, so their reported DualMS sub-times exclude that work but their signature sizes describe the full encoded object.

A.7 Version

Based on the LaTeX template for Artifact Evaluation V20220926.

[1] https://bridges.monash.edu/articles/Practice-Oriented_Techniques_in_Lattice-Based_Cryptography/12279728

[2] https://github.com/malb/lattice-estimator